A Strategic Analysis of Optimal Constellation Mapping (OCM) for the Maheshwara Sutras Phonemic Vector Space

Section 1: The Initial Script Constellation: A Foundational Analysis

This section formally defines the problem space by establishing the precise set of items to be mapped—the phonemes—and the mathematical space into which they are mapped—the 8-dimensional hypercube. These parameters form the baseline for the entire Optimal Constellation Mapping (OCM) protocol.

1.1 Defining the Mapping Set (N): The Unique Phonemes of the Maheshwara Sutras

The foundational element of the Octeract Cypher protocol is the set of N unique phonemes derived from Pāṇini's *Maheshwara Sutras* , also known as the *Śiva Sutras*. This set of 14 aphorisms is the pillar upon which Pāṇini's *Aṣṭādhyāyī*, the complete grammar of Sanskrit, rests. These verses provide a systematic organization of the Sanskrit phonemes (termed *Akṣara-samamnaya*, or "recitation of phonemes") for grammatical exposition.
The 14 *sutras* are :

1.
2.
3.
4.
5.
6.
7.
8.
9.
10.
11.
12.
13.
14.

The "script constellation" is formed from the unique phonemes *within* these sutras. It is critical to differentiate the phonemes from the final "dummy letters," or *anubandha*. These markers (conventionally uppercase: Ṇ, K, Ṅ, C, Ṭ, Ṇ, M, Ñ, Ṣ, Ś, V, Y, R, L) are not part of the phonemic inventory themselves; they are a meta-linguistic device used by Pāṇini to create *pratyāhāras*—concise codes for referring to specific groups of sounds.

Therefore, the set N for mapping *excludes* these 14 *anubandha*. The phoneme 'ha' appears twice (in *sutra* 5 and *sutra* 14), but is treated as a single, unique phoneme.
This enumeration results in a definitive set of **N=42** unique phonemes. This set comprises 9 vowels (the *svaras*: a, i, u, ṛ, ḷ, e, o, ai, au) and 33 consonants (the *vyañjanas*: ha, ya, va, ra, la, etc.). This set of 42 phonemes is the "facility" set (N=42) for the OCM protocol's Quadratic Assignment Problem.

Table 1: The Maheshwara Sutras Phoneme Set (N=42)

Sutra Number	Sutra Text (Phonemes)	Unique Phonemes Contributed
1	a i u	a, i, u
2	ṛ ḷ	ṛ, ḷ
3	e o	e, o
4	ai au	ai, au
5	ha ya va ra	ha, ya, va, ra
6	la	la
7	ña ma ṅa ṇa na	ña, ma, ṅa, ṇa, na
8	jha bha	jha, bha
9	gha ḍha dha	gha, ḍha, dha
10	ja ba ga ḍa da	ja, ba, ga, ḍa, da
11	kha pha cha ṭha tha ca ṭa ta	kha, pha, cha, ṭha, tha, ca, ṭa, ta
12	ka pa	ka, pa
13	śa ṣa sa	śa, ṣa, sa
14	ha	(No new phonemes; 'ha' is a duplicate)
Total		**42 Unique Phonemes**

1.2 Defining the Geometric Space: The 8-Dimensional Hypercube

The target space for the mapping is an 8-dimensional hypercube, or 8-cube, as defined by the protocol's "approved 8-feature schema." Each phoneme is mapped to an 8-bit binary vector.
A hypercube is a generalization of a 3-dimensional cube to a higher-dimensional space. The geometric structure of an n-dimensional hypercube is defined by its vertices and edges. An n-cube has 2^n vertices (corners). For the 8-dimensional space specified, there are 2^8 = 256 total available vertices, each corresponding to a unique 8-bit vector.
The "direct encoding" has mapped the N=42 phonemes to a specific *subset* of these 256 available vertices, creating the "initial 'script constellation'." The OCM protocol's task is to optimize the *assignment* of phonemes to these 42 selected vertices.
The "geometric distance" specified in the protocol is the **Hamming distance**. This is the theoretically sound and standard metric for this vector space. The Hamming distance between two binary vectors (strings) is the number of positions at which their corresponding symbols (bits) differ. For example, the distance between 01101000 and 00101100 is 2.
In computational phonology, this metric has a strong theoretical basis. When phonemes are represented as vectors of binary articulatory features (e.g., [+nasal], [-voiced]), the Hamming distance directly quantifies the number of features by which two sounds differ. This provides a robust, non-arbitrary measure of phonetic similarity. In the geometry of the hypercube, two vertices are connected by an edge if and only if their Hamming distance is 1.

1.3 The Nature of the Problem: Sparsity and the Two Sources of Structure

The parameters established in Sections 1.1 and 1.2 clarify the precise nature of the task and the strategic choice at hand.
First, the problem is *sparse*. We are assigning N=42 phonemes to a pre-selected set of 42 vertices ("locations") within a much larger space of M=256 total vertices. The OCM protocol is therefore a *standard* Quadratic Assignment Problem (QAP). It is not a selection-and-assignment problem (choosing the best 42 vertices out of 256). Rather, it is a *permutation* problem: finding the optimal 1-to-1 mapping, or arrangement, of the N=42 phonemes onto the N=42 existing vectors.
Second, the protocol presents a fundamental choice between two different sources of "structural meaning."

1. **Schema-Driven Structure (Path 2):** The "direct encoding" is not random. Its structure is a direct artifact of the "approved 8-feature schema." If Feature 1 (the first bit) corresponds to the phonological feature [+/-nasal], then all nasal phonemes (e.g., ña, ma, ṅa, ṇa, na) will be mapped to vectors that share the same value for that bit. Therefore, the "clusters, paths, and structures" that a Geometric Structural Analysis (GSA) would find in Path 2 are the *a priori*, *theoretical*, and *articulatory* structures imposed by the schema itself.
2. **Corpus-Driven Structure (Path 1):** This path seeks to *generate a new structure* based on *a posteriori*, *empirical* data. By analyzing a large Sanskrit corpus for phoneme co-occurrence , it builds a Sign Similarity Matrix (S) that reflects how phonemes are *actually used* in the language. The OCM optimization then *forces* the geometric map to conform to this *distributional* reality.

This is the central dilemma: the "Direct Analysis Path" (Pathway 2) is an analysis of the *theoretical schema's geometry*, while the "Corpus-Driven Path" (Pathway 1) is the *construction and optimization of an empirical, distributional geometry*. The decision is not merely "analysis vs. optimization" but a choice between two distinct phonological and computational philosophies.

Section 2: Pathway I: The Corpus-Driven Optimization (OCM)

This section provides a complete technical analysis of the recommended Pathway 1. It details the methodology for data ingestion (Module 1), similarity matrix generation (Module 2), and the execution of the OCM (QAP solved by SA).

2.1 Corpus Sourcing and Ingestion (Module 1)

Requirement: The protocol requires a "standardized digital corpus of Sanskrit text" from which to derive phoneme co-occurrence statistics.
Candidate Corpora: The field of computational Sanskrit provides several high-quality, programmatically accessible corpora:

1. **Digital Corpus of Sanskrit (DCS):** This is the foremost candidate. It is a large, "searchable collection of lemmatized Sanskrit texts" , comprising approximately 4.8 million manually tagged words. It is explicitly "designed for text-historical research in Sanskrit

linguistics and philology" and covers a vast temporal span from 500 BCE to 1900 CE. Crucially, it supports queries for "lexical units (words) and their collocations" and is available in an XML-wrapped format with full morphological and lexical analysis.

2. **Classical Language Toolkit (CLTK) Repositories:** The CLTK offers a suite of Sanskrit corpora , including sanskrit_text_dcs (a version of the above), sanskrit_text_gitasupersite , sanskrit_text_wikipedia, and sanskrit_text_sanskrit_documents. These are readily available for computational analysis.
3. **Sanskrit Heritage Site:** This resource provides a "Sanskrit Reader" that parses text and a digital version of the Monier-Williams dictionary, which could serve as a lexical base.

Corpus Selection Criteria and *Sandhi*: For the task of generating a *phoneme* similarity matrix, the choice of corpus is critical. The primary challenge in Sanskrit computational linguistics is *sandhi*, the phonological phenomenon where sounds are fused or changed at word boundaries. Analyzing a standard, *post-sandhi* text would yield deeply flawed statistics. For example, if word_A ends in a and word_B begins with i, the text may show them fused as e. A simple Grapheme-to-Phoneme (G2P) analysis would register the phonemes *within* e, entirely missing the *functional co-occurrence* of a and i across a word boundary.
This makes the "Sandhi-split" nature of the **Digital Corpus of Sanskrit (DCS)** its most compelling feature. The DCS is a "Sandhi-split corpus of Sanskrit texts". This means the computationally intensive work of "sandhi reversion" —recovering the "pre-sandhi" forms—has already been performed. Ingesting the DCS (via CLTK or directly) allows the framework to bypass this complex step and immediately analyze the true co-occurrence of phonemes at morpheme and word boundaries, which is a much more accurate measure of "distributional similarity."

2.2 Generation of the Sign Similarity Matrix (S) (Module 2)

Objective: To automatically generate the N \times N (42x42) Sign Similarity Matrix (S), where S[i,j] quantifies the "functional similarity" between phoneme i and phoneme j.
This task is accomplished by the "Language-Agnostic Profiling" (Module 2), which entails a multi-step computational linguistics pipeline:

1. **Grapheme-to-Phoneme (G2P) Conversion:** The ingested corpus (e.g., the DCS) is graphemic (written in, for example, Devanagari or ITRANS transliteration). This text must be converted into a "continuous stream of phonemes". This requires a standardized Sanskrit G2P converter, a common tool in computational phonology. As established, this G2P conversion must be applied to the *sandhi-split* (pre-sandhi) text to be effective.
2. **Context Definition:** The system must define what "appearing together" means. This is a critical parameter. The analysis can be configured to scan for co-occurrence within different windows:
    - **Phoneme N-grams:** This is the most direct method. It involves analyzing contiguous sequences of phonemes. A 5-gram window (n=5) has been shown to be highly effective in capturing the phonemic context for Vedic Sanskrit sandhi and is a strong candidate. * **Lexical Co-occurrence:** Analyzing phoneme co-occurrence within the same *pada* (word) or stem. Tools like PhonMatrix analyze co-occurrence within a specified context (e.t., VCV).
    - **Syllabic Co-occurrence:** Using a syllabification algorithm (provided by CLTK) to analyze which phonemes co-occur within the same syllable.
3. **Contingency Table Generation:** Based on the chosen context (e.g., a 5-gram sliding window), the framework iterates through the entire phonemic corpus. It populates a 42x42

matrix of raw co-occurrence counts, which serves as the "contingency table" for statistical analysis.

4. **Application of Association Measures:** Raw co-occurrence counts are heavily biased by the high-frequency phonemes (e.g., a). A statistically "interesting" co-occurrence is one that happens *more frequently than expected by chance*. The raw counts must be normalized using a statistical association measure. * **Pointwise Mutual Information (PMI):** This is an excellent candidate. The DCS's own documentation notes the use of "mutual information" to calculate the "interestingness" of bigrams, as it "emphasizes a combination if its components occur together in most cases". PMI is also cited as a method for calculating phonetic similarity.
    - **\varphi (Phi) Coefficient:** The PhonMatrix tool recommends the \varphi value, a normalized \chi^2 measure, as "especially useful for pattern detection" because it ranges from -1 to 1 and effectively handles the full data in the contingency table.
    - **Other Measures:** Cosine similarity , Jaccard, Dice , and cross-entropy are also standard measures.

Final Output (S): The final output of Module 2 is the N \times N Sign Similarity Matrix, S, where S[i,j] is the calculated association score (e.g., PMI) between phoneme i and phoneme j. This matrix S becomes the "Flow Matrix" for the QAP.

2.3 The Optimization Framework: OCM as a Quadratic Assignment Problem (QAP)

The protocol correctly identifies the OCM as a Quadratic Assignment Problem (QAP). The QAP is a fundamental problem in combinatorial optimization, famously NP-hard.
The problem is defined by two N \times N matrices, A and B. The objective is to find the permutation p of N items that minimizes the total cost. In the Koopmans-Beckmann formulation, the cost is:
For the OCM protocol, the components are formally defined as follows:

- **N[span_34](start_span)[span_34](end_span)[span_36](start_span)[span_36](end_span)**: The size of the problem, N=42, representing the 42 unique phonemes.
- **i, j**: Indices from 1 to 42, representing two *phonemes* (e.g., i = 'ka', j = 'pa').
- **p(i), p(j)**: The *locations* (8-bit vectors) assigned to phonemes i and j. These are indices from 1 to 42, representing the set of 42 pre-selected vectors.
- **A (Flow Matrix)**: This is our N \times N **Sign Similarity Matrix (S)** generated in Section 2.2. A[i,j] is the high-level (e.g., PMI) *functional similarity* between phoneme i and phoneme j.
- **B (Distance Matrix)**: This is an N \times N matrix of the **Hamming distances** between the 42 8-bit vectors. B[p(i), p(j)] is the *geometric distance* between the vector assigned to phoneme i and the vector assigned to phoneme j.

This formulation can be conceptualized as a graph matching problem. The OCM seeks to find the optimal alignment between two graphs:

1. **Graph A (Functional Graph):** The 42 phonemes as nodes, with edge weights A[i,j] representing their functional similarity.
2. **Graph B (Geometric Graph):** The 42 8-bit vectors as nodes, with edge weights B[k,l] representing their Hamming distance.

The OCM's objective is to find the permutation p that maps the nodes of Graph A to Graph B to minimize the "cost"—in this case, ensuring that high-similarity edges in Graph A are mapped to

low-distance (low-Hamming) edges in Graph B.

Table 3: QAP Formulation for Phoneme-Vector Mapping

QAP Term	Protocol Component	Description (Source)
Facilities / Agents (N)	42 Unique Phonemes	The set N from the *Maheshwara Sutras*.
Locations (N)	42 8-bit Vectors	The "direct encoding" vertices selected from the 8-D hypercube.
Flow Matrix (A)	N \times N Sign Similarity Matrix (S)	Generated from corpus co-occurrence (e.g., PMI).
Distance Matrix (B)	N \times N Hamming Distance Matrix	Pre-calculated Hamming distances between all pairs of the 42 8-bit vectors.

2.4 Mapping Functional Similarity to Geometric Distance (Hamming)

The core objective of this QAP formulation is to enforce a well-established principle in cognitive science and linguistics: the correlation between form and function (or meaning). Research has shown that across languages, "more semantically similar word pairs are also more phonologically similar". Other work extends this principle, showing that conceptual similarity (whether semantic or phonological) is represented in an "abstract cognitive space" where distance is a metaphor for dissimilarity.

The OCM protocol is a direct, computational implementation of this principle. It is designed to create an "abstract cognitive space" (the 8-D hypercube) where the *geometric distance* (Hamming) is *not* arbitrary, but is instead optimized to reflect *functional similarity* (corpus-derived PMI).

The QAP cost function Cost(p) = \Sigma \Sigma (A[i,j] \cdot B[p(i), p(j)]) is a global measure of "structural dissonance." A high cost means that phonemes with high similarity (high A[i,j]) are being assigned to vectors with high dissimilarity (high B[p(i), p(j)]), breaking the desired correlation. The optimization's goal is to find the permutation p that *minimizes* this dissonance, thus creating the most "structurally meaningful" map.

The use of Hamming distance is essential. As a metric that counts the number of differing binary features , it is the *native* and theoretically-grounded metric for the binary vector space created by the 8-feature schema.

2.5 Heuristic Solution via Simulated Annealing (SA)

The QAP is NP-hard. An exhaustive search of all possible permutations (N!, or 42!) is computationally impossible. Therefore, the protocol's mandate for a metaheuristic is necessary. The chosen heuristic is **Simulated Annealing (SA)**, a method extensively and successfully applied to the QAP. SA is an adaptation of the Metropolis-Hastings algorithm , which mimics the process of annealing in metallurgy: a material is heated and then slowly cooled to allow its molecular structure to settle into a low-energy, minimal-defect state.

The SA algorithm for the OCM-QAP will proceed as follows:

1. **Initialization:** The algorithm begins with an initial solution, p_{current}, which is the "direct encoding" map. A high initial "temperature," T, is set. The total cost of the initial map, C_{current}, is calculated using the QAP cost function.
2. **Neighbor Generation:** A "neighboring state," p_{neighbor}, is generated. For the QAP,

this is typically done by selecting two random indices (i, j) and *swapping* their assignments: $p_{neighbor} = swap(p_{current}, i, j)$.

3. **Cost Evaluation:** The cost of the new arrangement, $C_{neighbor}$, is calculated. The change in cost is $\Delta C = C_{neighbor} - C_{current}$.
4. **Acceptance Criterion (Metropolis Criterion):**
    - **If $\Delta C < 0$:** The new state is *better* (has a lower cost). The algorithm *always* accepts the move: $p_{current} = p_{neighbor}$.
    - **If $\Delta C > 0$:** The new state is *worse* (has a higher cost). The algorithm *may still accept the move* with a probability $P = \exp(-\Delta C / T)$.
5. **The Key Advantage:** This probabilistic acceptance of "uphill" or "worse" moves is the central feature of SA. A simple greedy algorithm would get stuck in the first "local minimum" it finds. SA's ability to take worse steps allows it to "jump" out of these local minima and explore the broader solution space, making it far more likely to find a high-quality, near-global optimum.
6. **Cooling and Termination:** After a set number of iterations, the temperature T is lowered according to a "cooling schedule". As T approaches 0, the probability of accepting "worse" moves decreases, and the algorithm "freezes" into a final, stable, near-optimal solution.

2.6 Anticipated Outcome: The Structurally Meaningful Constellation

The final output of Pathway 1 will be the optimized permutation vector $p_{optimal}$. This vector defines the new, optimized "script constellation."

This constellation will be "structurally meaningful" in a profound, *distributional* sense. Phonemes that frequently co-occur, interact, or appear in similar contexts in the vast corpus of Sanskrit literature (e.g., stops and their corresponding nasals, or vowels that participate in the same *sandhi* rules) will be assigned to 8-bit vectors that are geometrically close in the hypercube (i.e., have a low Hamming distance).

A subsequent Geometric Structural Analysis (GSA) performed on *this* optimized map would reveal clusters and pathways based on *empirical linguistic function and usage*, not merely on the *a priori* theoretical categories of the 8-feature schema.

Section 3: Pathway II: Direct Geometric Structural Analysis (GSA)

This section provides a technical analysis of the alternative, faster Pathway 2. This path bypasses the OCM optimization and proceeds directly to analyzing the "direct encoding" map.

3.1 Analysis of the Unoptimized "Direct Encoding" Map

Pathway 2 accepts the initial "direct encoding" or "first draft" as the final constellation. This map is not a random assignment; it is a direct, 1-to-1 unpermuted representation of the "approved 8-feature schema."

Analyzing a "direct encoding" is a common practice in data science, often used to understand the properties of the encoding scheme itself. Unlike deep learning models that generate *learned* embeddings , this "direct encoding" is static. The insights gained from a GSA on this map are not about the statistical behavior of Sanskrit phonemes, but about the *inherent geometric structure of the 8-feature schema*.

As established (Section 1.3), the GSA in this pathway will not *discover* new linguistic regularities from data. It will *reveal* the theoretical regularities, assumptions, and latent structures of the 8-feature schema that was used to create the map. For instance, if the schema is based on standard phonological features , the GSA will find clusters corresponding to "vowels," "nasals," "stops," etc., simply because they were *encoded* with similar 8-bit vectors.

3.2 GSA Protocols for High-Dimensional Hypercubes

The objective of Pathway 2 is to analyze the "clusters, paths, and structures" within the 8-dimensional, 42-point "direct encoding" constellation.
Geometric Structural Analysis (GSA) (or, more broadly, *Computational Geometry*) is the branch of computer science devoted to algorithms that analyze geometric structures. In this context, it combines with Topological Data Analysis to find meaningful patterns in high-dimensional data.
The key protocols for this analysis would include:

1. **Dimensionality Reduction (PCA):** The 8-dimensional space is unvisualizable. The primary method for projecting this data into 2D or 3D is **Principal Component Analysis (PCA)**. PCA finds the orthogonal axes (principal components) that capture the *maximum variance* in the 42-point dataset. This provides the most "faithful" low-dimensional visualization of the 8-D structure.
2. **PCA on Hypercubes:** Applying PCA to hypercubic binary state spaces is a known and powerful technique. As S_ notes, PCA is highly effective at "capturing polarized distributions" and "discovering hidden patterns" in such data. By projecting the hypercubic vertices and edges, PCA can reveal "pathways of correlated spin flips" —which, in this context, are *paths of single-feature-changes* (Hamming distance 1) that connect different phoneme classes.
3. **Cluster Analysis:** In either the full 8-D space or the projected PCA space, clustering algorithms can be applied to identify "dense" groupings of phonemes. While standard methods like K-Means or DBSCAN are common, more physics-inspired metaheuristics like the **Gravitational Search Algorithm (GSA)** can also be used for data clustering.
4. **Path and Adjacency Analysis:** The analysis can also explore the graph-theoretic structure of the constellation. This involves finding shortest paths (in Hamming distance) between phonemes and analyzing the "neighborhood" of each phoneme (the set of phonemes at Hamming distance 1). This would reveal the single-feature changes that "flip" one phoneme into another, as defined by the 8-feature schema.

3.3 Potential Artifacts and Insights from a Non-Optimal Map

This pathway is not without value, but its risks must be clearly understood.
Primary Risk: Tautological Insights. The most significant risk is that the GSA will produce *tautological* findings. As described in 3.1, the GSA *will* find clusters. However, these clusters are simply an artifact of the *a priori* schema. If the schema defines 'a', 'i', and 'u' as [+syllabic, -consonantal...], the GSA will "discover" a "vowel cluster." This is a circular finding: the analysis is merely reporting the structure of its own input assumptions (the schema). It provides no external validation or new knowledge about the Sanskrit language itself.
Potential Insight: Schema Validation. Pathway 2 is not, therefore, an analysis of Sanskrit phonology. It is a *validation of the 8-feature schema itself*. It provides the answer to a different, but still useful, question: "Is the 'approved' 8-feature schema a *geometrically coherent* model of

phonology?"
For example, a "good" schema would naturally place articulatorily similar phonemes (like the labials 'pa', 'ba', and 'ma') into a "tight" cluster (low mutual Hamming distance) in the 8-D space. A "bad" or "incoherent" schema might scatter them across the hypercube. Pathway 2 is the diagnostic tool to determine this.
The Trade-Off: The choice is one of

- **Pathway 2:** Fast. Delivers immediate insights into the *theoretical model* (the schema).
- **Pathway 1:** Slow and computationally expensive. Delivers *new, empirical insights* into the *language itself* (the corpus).

Section 4: Comparative Analysis and Strategic Recommendation

This final section synthesizes the technical analyses from Sections 2 and 3 to provide a formal, actionable recommendation for the OCM protocol.

4.1 Structural Integrity vs. Analytical Velocity: A Comparative Assessment

The decision between the two pathways is a classic trade-off between analytical velocity and structural integrity. The protocol's stated goal is to find the "most 'optimal' arrangement" and a "structurally meaningful" map, with the explicit caveat that Pathway 2 may be "less 'structurally meaningful'". This framing implies that the project's ultimate aim is an *empirically-grounded* map, not just a *theoretically-convenient* one.

- **Pathway I (Corpus-Driven OCM):**
  - **Integrity:** Very High. The resulting constellation is mathematically optimized (via QAP+SA) to reflect *empirical linguistic reality* as captured by a large, objective corpus. The "structure" is *emergent* from data, validated, and robust.
  - **Velocity:** Slow. This path requires multiple, non-trivial computational steps: (1) Corpus ingestion and processing, (2) G2P/Sandhi-reversal, (3) Corpus-wide N-gram statistical analysis to build S, and (4) A computationally expensive SA run to solve an NP-hard problem.
- **Pathway II (Direct GSA):**
  - **Integrity:** Unknown and Schema-Dependent. The map is "meaningful" *only if* the 8-feature schema is a robust, well-designed phonological model. The GSA reveals the *schema's* structure, not the *language's* structure. It risks analyzing mere artifacts of the "direct encoding".
  - **Velocity:** Fast. The "direct encoding" is already complete. The GSA (PCA, clustering) is a standard, computationally inexpensive procedure.

This comparison is formalized in the decision matrix below.
Table 4: Decision Matrix: Corpus-Driven OCM vs. Direct GSA

Criterion	Pathway I (Corpus-Driven OCM)	Pathway II (Direct GSA)
Structural Integrity	**Very High.** Optimized to reflect *empirical* linguistic function.	**Unknown.** Reflects *theoretical* schema. Risks analyzing encoding artifacts.

Criterion	Pathway I (Corpus-Driven OCM)	Pathway II (Direct GSA)
Source of "Meaning"	*A posteriori* (Distributional, from corpus data)	*A priori* (Theoretical, from 8-feature schema)
	Computational Cost	**High.** Corpus parsing + G2P + NP-hard optimization (QAP+SA).
Time-to-Insight	**Slow.** Requires execution of Modules 1 & 2 (Corpus) and OCM (SA).	**Fast.** Proceeds immediately to analysis.
Primary Output	A *validated, optimized* phoneme constellation.	An *analysis* of the *unoptimized "draft"* constellation.

4.2 Recommendation for Protocol Execution

The analysis confirms that the "Recommended" tag on Pathway 1 in the initial query is well-founded.
Pathway 2 represents a "false economy." While faster, it is an insular exercise. It delivers a comprehensive analysis of the "approved 8-feature schema," but it provides no external validation. It cannot, by definition, produce the "most 'optimal' arrangement" because it *bypasses* optimization.
Pathway 1, in contrast, is the only method that breaks out of this theoretical insularity. It uses the digital Sanskrit corpus as an external, objective source of truth. It then uses the OCM (QAP+SA) to *force* the geometric constellation to align with the statistical realities of the Sanskrit language itself. This is the only path that produces a "mathematically robust constellation" that is also *linguistically* robust and meaningful.
Final Recommendation:
It is definitively recommended to **proceed with Pathway 1: The Corpus-Driven Path.**

1. Authorize the execution of **Module 1 (Corpus Ingestion)**. The **Digital Corpus of Sanskrit (DCS)** should be the primary target, specifically for its size, lemmatization, and critical "Sandhi-split" annotations.
2. Authorize the execution of **Module 2 (Language-Agnostic Profiling)**. This module must be configured to:
    - Use the *sandhi-split* text for its Grapheme-to-Phoneme (G2P) pipeline.
    - Generate the 42x42 co-occurrence matrix using a context window (e.g., 5-grams) or lexical co-occurrence.
    - Calculate the final association scores for the Sign Similarity Matrix (S) using a robust measure such as **Pointwise Mutual Information (PMI)**. 3. Upon the successful generation of the Sign Similarity Matrix (S), the OCM protocol's dependency will be met, and the **Simulated Annealing (SA)** algorithm can be executed to find the p_{optimal} permutation.

This is the only pathway that fulfills the protocol's stated objective of discovering a "most 'optimal' arrangement" that is verifiably "structurally meaningful."

Works cited

1. Mnemonics in Sanskrit | College Hive, https://notes.collegehive.in/books/indian-knowledge-system/page/mnemonics-in-sanskrit 2.

Shiva Sutras - Wikipedia, The Free Encyclopedia | PDF | Phonetics - Scribd, https://www.scribd.com/document/180592053/Shiva-Sutras-Wikipedia-The-Free-Encyclopedia 3. An Introduction to Shiva Sutras of Vasugupta | by Merging Into Oneness - Medium, https://mergingintooneness.medium.com/shiva-sutras-vs-shiva-stotras-a-beginners-guide-8365fca37148 4. chapter 1 - the alphabet, https://pub-f6bbf473991b4da884ae602cdac531e9.r2.dev/Shastras/Books/English/AdiLaghu-Part-1-1.pdf 5. The “Shiva Sutras” | glossologue - WordPress.com, https://glossologue.wordpress.com/2017/05/13/the-shiva-sutras/ 6. Sanskrit Alphabet Shiva Sutras (Maheshvara Sutrani) - The Matheson Trust, https://www.themathesontrust.org/library/shiva-sutras 7. Shiva Sutras - Wikipedia, https://en.wikipedia.org/wiki/Shiva_Sutras 8. An Analysis of Hypercube Architectures for Image Pattern Recognition Algorithms - University of Michigan, http://tnm.engin.umich.edu/wp-content/uploads/sites/353/2017/12/1987.01.An-Analysis-Of-Hypercube-Architectures-For-Image-Pattern-Recognition-Algorithms_SPIE.pdf 9. Hamming Distance Explained: The Theory and Applications - DataCamp, https://www.datacamp.com/tutorial/hamming-distance 10. Phonetic alignment and similarity - Department of Computing Science, https://webdocs.cs.ualberta.ca/~kondrak/papers/chum.pdf 11. PWESuite: Phonetic Word Embeddings and Tasks They Facilitate - arXiv, https://arxiv.org/html/2304.02541 12. Phonological Similarity and Cognate Detection - GitHub Pages, https://dmort27.github.io/subwordmodeling/lectures/23-phonological-similarity.pdf 13. Hypercube Graphs and the Representation Theory of Symmetric Groups An Honors College Project Presented t, https://educ.jmu.edu/~duceyje/undergrad/2022/sherwocj_project.pdf 14. Quantum Speedup for the Quadratic Assignment Problem - arXiv, https://arxiv.org/pdf/2410.12181 15. Fast Approximate Quadratic Programming for Graph Matching ..., https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0121002 16. Beyond graphemes and phonemes: continuous phonological features in neural text-to-speech synthesis - ISCA Archive, https://www.isca-archive.org/interspeech_2024/tannander24_interspeech.pdf 17. Binary Phoneme Classification Using Fixed and Adaptive Segment- Based Neural Network Approach, https://www.ccis2k.org/iajit/PDF/vol.8,no.1/9.pdf 18. Digital Corpus of Sanskrit (DCS) - Online Sanskrit dictionary and ..., http://www.sanskrit-linguistics.org/dcs/ 19. Lexical n-grams - Digital Corpus of Sanskrit (DCS) - Online Sanskrit dictionary and annotated corpus, http://www.sanskrit-linguistics.org/dcs/index.php?contents=help_ngrams 20. PhonMatrix: Visualizing co-occurrence constraints ... - ACL Anthology, https://aclanthology.org/P13-4013.pdf 21. Improving Cross-Lingual Phonetic Representation of Low-Resource Languages Through Language Similarity Analysis This work was supported by the National Research Foundation of Korea grant funded by the Korea government (MSIT). (No. 2021R1A2C1014044) - arXiv, https://arxiv.org/html/2501.06810v1 22. arXiv:2410.22906v1 [cs.CL] 30 Oct 2024, https://arxiv.org/pdf/2410.22906 23. cltk/sanskrit_text_dcs: Sanskrit Corpus - GitHub, https://github.com/cltk/sanskrit_text_dcs 24. AWOL Index: DCS: Digital Corpus of Sanskrit, https://isaw.nyu.edu/publications/awol-index/html/kjc-fs-cluster.kjc.uni-heidelberg.de/dcs-index-php.html 25. Digital Corpus of Sanskrit - Computational Linguistics Lab, https://bond-lab.github.io/Corpus-Linguistics/pdf/corpora/2015-DigitalSanskritCorpus.pdf 26. Sanskrit — Classical Language Toolkit documentation, https://v0.cltk.org/en/latest/sanskrit.html 27. The Classical Language Toolkit (CLTK) — Classical Language Toolkit documentation, https://v0.cltk.org/ 28. cltk/sanskrit_text_gitasupersite: sanskrit monolingual corpus - GitHub, https://github.com/cltk/sanskrit_text_gitasupersite 29. The Sanskrit Heritage Site, https://sanskrit.inria.fr/ 30. N-gram-Based Preprocessing for Sandhi Reversion in Vedic Sanskrit

- ACL Anthology, https://aclanthology.org/2024.nlp4dh-1.26.pdf 31. Investigating the universality of consonant and vowel co-occurrence restrictions | Glossa, https://www.glossa-journal.org/article/id/9373/ 32. [2301.12969] Using n-aksaras to model Sanskrit and Sanskrit-adjacent texts - arXiv, https://arxiv.org/abs/2301.12969 33. N-GRAM BASED ALGORITHM FOR DISTINGUISHING BETWEEN HINDI AND SANSKRIT TEXTS, http://www.ir.juit.ac.in:8080/jspui/bitstream/123456789/6742/1/N%20Gram%20Based%20Algorithm%20for%20Distinguishing%20Between%20Hindi%20and%20Sanskrit%20Texts.pdf 34. Building a text retrieval system for the Sanskrit language: Exploring indexing, stemming, searching issues | Request PDF - ResearchGate, https://www.researchgate.net/publication/369952207_Building_a_text_retrieval_system_for_the_Sanskrit_language_Exploring_indexing_stemming_searching_issues 35. A MEASURE OF PHONETIC SIMILARITY TO QUANTIFY PRONUNCIATION VARIATION BY USING ASR TECHNOLOGY, https://www.gavo.t.u-tokyo.ac.jp/~mine/paper/PDF/2015/ICPHS_t2015-8.pdf 36. Quantifying and Reducing Speaker Heterogeneity within the Common Voice Corpus for Phonetic Analysis - arXiv, https://arxiv.org/html/2506.00733v1 37. The Classical Language Toolkit: An NLP Framework for Pre-Modern Languages, https://www.researchgate.net/publication/353492044_The_Classical_Language_Toolkit_An_NLP_Framework_for_Pre-Modern_Languages 38. The N-Grams Based Text Similarity Detection Approach Using Self-Organizing Maps and Similarity Measures - MDPI, https://www.mdpi.com/2076-3417/9/9/1870 39. Measuring phonological distance between languages - UCL Discovery, https://discovery.ucl.ac.uk/10058348/1/Eden_10058348_thesis.pdf 40. [2509.04089] Gromov-Wasserstein and optimal transport: from assignment problems to probabilistic numeric - arXiv, https://arxiv.org/abs/2509.04089 41. A New Genetic Algorithm for the Quadratic Assignment Problem - ResearchGate, https://www.researchgate.net/publication/220668775_A_New_Genetic_Algorithm_for_the_Quadratic_Assignment_Problem 42. [2007.14885] Performance Analysis of Meta-heuristic Algorithms for a Quadratic Assignment Problem - arXiv, https://arxiv.org/abs/2007.14885 43. Generalizing Graph Matching beyond Quadratic Assignment Model - NIPS papers, http://papers.neurips.cc/paper/7365-generalizing-graph-matching-beyond-quadratic-assignment-model.pdf 44. Graph matching based on similarities in structure and attributes - arXiv, https://arxiv.org/html/2409.20212v1 45. Semantic Sentence Matching via Interacting Syntax Graphs - ACL Anthology, https://aclanthology.org/2022.coling-1.78.pdf 46. Wordform Similarity Increases With Semantic Similarity: An Analysis of 100 Languages - colala, https://colala.berkeley.edu/papers/dautriche2016wordform.pdf 47. Phonological similarity between words is represented in declarative memory as spatial distance - PMC - NIH, https://pmc.ncbi.nlm.nih.gov/articles/PMC10197043/ 48. An Approach for Controlled Random Tests with a Given Hamming Distance Generation, https://www.mdpi.com/2076-3417/15/18/9951 49. Comparative Performance of Tabu Search and Simulated Annealing ..., https://arxiv.org/pdf/1010.0157 50. [1111.1353] An efficient implementation of the simulated annealing heuristic for the quadratic assignment problem - arXiv, https://arxiv.org/abs/1111.1353 51. [1010.0157] Comparative Performance of Tabu Search and Simulated Annealing Heuristics for the Quadratic Assignment Problem - arXiv, https://arxiv.org/abs/1010.0157 52. An overview of some heuristic algorithms for combinatorial optimization problems - SciSpace, https://scispace.com/pdf/an-overview-of-some-heuristic-algorithms-for-combinatorial-57j3rdtgqf.pdf 53. Heuristics for the Quadratic Assignment Problem (QAP), https://cran.r-project.org/web/packages/qap/qap.pdf 54. Simulated annealing for optimization - MATLAB Answers - MathWorks,

https://www.mathworks.com/matlabcentral/answers/463547-simulated-annealing-for-optimizatio
n 55. Simulated annealing - Wikipedia, https://en.wikipedia.org/wiki/Simulated_annealing 56.
Simulated Annealing : Methods and Real-World Applications – OMSCS 7641,
https://sites.gatech.edu/omscs7641/2024/02/19/simulated-annealing-methods-and-real-world-ap
plications/ 57. Graph Search Algorithms 3: Simulated Annealing - Allison Liu,
https://allisonznliu.com/simulated-annealing 58. 11.1.4 Simulated Annealing - The Netlib,
https://www.netlib.org/utk/lsi/pcwLSI/text/node252.html 59. What makes Simulated Annealing
more likely in finding a more-optimal solution than Hill-Climbing? - Stack Overflow,
https://stackoverflow.com/questions/78481558/what-makes-simulated-annealing-more-likely-in-fi
nding-a-more-optimal-solution-th 60. VectorNet: Encoding HD Maps and Agent Dynamics from
Vectorized Representation - Waymo,
https://waymo.com/intl/fil/research/vectornet-encoding-hd-maps-and-agent-dynamics-from-vecto
rized-representation 61. Encoding Geospatial Vector Data for Deep Learning: LULC as a Use
Case - MDPI, https://www.mdpi.com/2072-4292/14/12/2812 62. Raster versus Vector Data
Encoding and Handling: A Commentary - ASPRS,
https://www.asprs.org/wp-content/uploads/pers/1987journal/oct/1987_oct_1397-1398.pdf 63.
MetaStore: Analyzing Deep Learning Meta-Data at Scale - VLDB Endowment,
https://www.vldb.org/pvldb/vol17/p1446-cao.pdf 64. arXiv:2501.03304v2 [cs.RO] 8 Jan 2025,
https://arxiv.org/pdf/2501.03304 65. Computational geometry - Wikipedia,
https://en.wikipedia.org/wiki/Computational_geometry 66. Computational Geometry: Algorithms
& Uses | StudySmarter,
https://www.studysmarter.co.uk/explanations/math/geometry/computational-geometry/ 67.
Computational Geometry and Topology for Data Analysis - Inria,
https://team.inria.fr/geometrica/files/2016/01/main-1.pdf 68. (PDF) Geometric Structure of
High-Dimensional Data and Dimensionality Reduction,
https://www.researchgate.net/publication/266375900_Geometric_Structure_of_High-Dimension
al_Data_and_Dimensionality_Reduction 69. Geometric Structure of High-Dimensional Data and
Dimensionality Reduction (English Version) (Chinese Edition) 7040317044, 9787040317046 -
DOKUMEN.PUB,
https://dokumen.pub/geometric-structure-of-high-dimensional-data-and-dimensionality-reduction
-english-version-chinese-edition-7040317044-9787040317046.html 70. Orthogonal projections
of hypercubes | Phys. Rev. E, https://link.aps.org/doi/10.1103/v291-9hxy 71. A Review on
Gravitational Search Algorithm and its Applications to Data Clustering & Classification -
ResearchGate,
https://www.researchgate.net/publication/276231583_A_Review_on_Gravitational_Search_Algo
rithm_and_its_Applications_to_Data_Clustering_Classification 72. (PDF) Binary Gravitational
Subspace Search for Outlier Detection in High Dimensional Data Streams - ResearchGate,
https://www.researchgate.net/publication/363468902_Binary_Gravitational_Subspace_Search_f
or_Outlier_Detection_in_High_Dimensional_Data_Streams